



RepoAI: Automated code refactoring through multi-agent LLM orchestration and retrieval-augmented generation

Nacha Chondamrongkul*, Moe Pyae Pyae Kyaw, Soe Moe Ko, Pyae Phyoo Paing, Min Khant Than Swe, Tew Hongthong

School of Applied Digital Technology, Mae Fah Luang University, Chiang Rai, Thailand

ARTICLE INFO

Keywords:

Code refactoring
Large language models
Multi-agent systems
Retrieval-augmented generation
Software maintenance
Github automation

ABSTRACT

While recent agentic coding tools have extended Large Language Model capabilities to repository-level tasks, their closed-source architectures preclude independent evaluation of pipeline components, cross-model comparison, and detection of self-evaluation bias. This paper presents RepoAI, an open, model-agnostic multi-agent system that automates repository-level code refactoring through coordinated LLM orchestration with cross-family validation. Our approach employs specialised agents for intent classification, code generation, and validation, working collaboratively to handle complex multi-file modifications. The system combines Retrieval-Augmented Generation for context-aware code understanding with a comprehensive validation pipeline that uses cross-family LLM judges to eliminate self-evaluation bias. We evaluate RepoAI on 1404 benchmark tasks using RefactorBench [1] (100 tasks from 10+ open-source projects), comparing five pipeline conditions across seven generator models from four model families. A Friedman test confirms statistically significant differences among conditions ($\chi^2 = 11.08$, $p = 0.026$), with pairwise Wilcoxon signed-rank tests showing that the full multi-agent pipeline significantly outperforms direct generation ($p = 0.007$). Multi-model evaluation demonstrates generalisability across Qwen, Devstral, Llama 4, and Gemma 2, while a systematic error taxonomy reveals that all 170 observed failures stem from model-specific generation errors rather than pipeline deficiencies.

1. Introduction

The rapid growth of large-scale software systems has created an urgent need for intelligent tools that support code comprehension, refactoring, and maintenance. Modern developers frequently interact with unfamiliar repositories, navigate complex directory structures, and attempt to modify codebases without breaking existing functionality. These tasks are not only time-consuming but also highly error-prone, particularly when developers must work across multiple files, resolve inter-file dependencies, or maintain consistent architectural patterns. As software development teams increasingly rely on distributed, collaborative workflows, the demand for automated assistance in repository understanding and safe code transformation continues to rise.

Recent advancements in Large Language Models (LLMs) have demonstrated strong capabilities in reasoning over source code, generating new functions, and providing natural-language explanations of technical concepts [2]. A new generation of agentic coding tools—including GitHub Copilot Workspace, Cursor, and OpenAI Codex—has begun to extend these capabilities to repository-level

* Corresponding author.

E-mail addresses: nacha.cho@mfu.ac.th (N. Chondamrongkul), 6531503158@lamduan.mfu.ac.th (M.P.P. Kyaw), 6531503185@lamduan.mfu.ac.th (S.M. Ko), 6531503175@lamduan.mfu.ac.th (P.P. Paing), 6531503154@lamduan.mfu.ac.th (M.K. Than Swe), tew.hon@mfu.ac.th (T. Hongthong).

<https://doi.org/10.1016/j.scico.2026.103477>

Received 28 January 2026; Received in revised form 23 March 2026; Accepted 26 March 2026

Available online 31 March 2026

0167-6423/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

operations, offering multi-file navigation, terminal access, and version control integration. However, these tools are predominantly proprietary, closed-source systems whose internal architectures, validation mechanisms, and failure modes are not publicly documented or independently evaluable. Crucially, they lack systematic mechanisms to detect self-evaluation bias when the same model family both generates and assesses code quality; moreover, no published study provides rigorous multi-model comparison or statistical ablation of their pipeline components. These gaps motivate an open, model-agnostic architecture that enables transparent evaluation across LLM families with principled validation.

1.1. Challenges in LLM-based code refactoring

Several challenges must be addressed to enable effective LLM-based code refactoring at the repository level. The most fundamental challenge concerns context limitation. Even the largest available context windows—up to 2 million tokens in Gemini 2.5 Pro—can accommodate only approximately 200,000 lines of code, given Python’s typical density of approximately 10 tokens per line. Real enterprise codebases and popular open-source projects routinely exceed this scale by orders of magnitude: the Linux kernel contains 40 million lines (400M tokens), while typical enterprise monorepos span hundreds of thousands of files. Moreover, empirical studies show that simply enlarging the context window does not resolve the problem. Liu et al. [3] demonstrate a U-shaped accuracy curve in which LLM performance degrades significantly when relevant content appears in the middle of a long context. Chroma Research’s “Context Rot” study [4] finds that degradation begins at approximately 50,000 tokens even within a 1-million-token window, a phenomenon driven by attention dilution and distractor interference. Without mechanisms to selectively retrieve relevant code segments, the model lacks sufficient grounded context to reason about cross-file dependencies and architectural patterns [5,6]. This limitation is compounded by the challenge of multi-file coordination, as repository-level refactoring often requires coordinated modifications across multiple files while maintaining consistency in imports, function signatures, and module interfaces. While recent agentic tools address this through iterative tool use, the question of whether multi-agent specialisation provides measurable quality gains over single-agent approaches remains open and is empirically under-studied [2].

Beyond coordination, validation and correctness present significant obstacles. LLMs may generate syntactically valid but semantically incorrect code, or may produce outputs that fail to satisfy the user’s original intent, making comprehensive validation mechanisms essential for detecting such errors before deployment [7,8]. Closely related is the challenge of hallucination and reliability: LLMs may produce plausible but incorrect code, particularly when operating without grounded context from the actual repository [9,10], necessitating retrieval mechanisms that anchor generation in actual code artefacts. Finally, integration with development workflows requires seamless connection with version control systems, including branch management, commit generation, and pull request creation, while maintaining audit trails for review.

1.2. Contributions

To address the limitations of existing code assistants, we present **RepoAI**, a web-based intelligent system that combines Retrieval-Augmented Generation (RAG), multi-agent orchestration, and comprehensive validation for automated code refactoring. The contributions of this work span architectural design, methodological innovation, and empirical investigation.

First, we introduce a multi-agent architecture built on Microsoft AutoGen that coordinates specialised agents for distinct phases of the refactoring workflow. The architecture separates concerns across agents responsible for intent classification, repository analysis, multi-file code generation, validation orchestration, and GitHub integration, enabling independent optimisation of each component while maintaining coherent end-to-end operation.

Second, we develop a RAG pipeline specifically designed for code repositories that constructs vectorised knowledge spaces from GitHub repositories using state-of-the-art embedding models. This pipeline enables context-aware retrieval of relevant code segments during generation, grounding LLM outputs in actual repository content and substantially reducing hallucination compared with retrieval-free approaches.

Third, we present a comprehensive validation framework that addresses the reliability challenges inherent in LLM-based code generation. The framework combines complementary validation strategies, including syntax checking through language-specific parsers, semantic preservation analysis comparing structural elements between the original and modified code, embedding-based similarity scoring for intent alignment assessment, and LLM-based verification that evaluates whether generated code fulfils user requirements.

Fourth, we implement automated GitHub integration that bridges the gap between code generation and deployment within standard development workflows. This integration handles the complete lifecycle from authenticated repository access through branch creation, file modification, and pull request generation with appropriate metadata for human review.

Fifth, we introduce a cross-family judge validation mechanism that eliminates self-evaluation bias by ensuring that the LLM evaluating generated code always belongs to a different model family from the generator. This design enables unbiased intent verification scoring and is validated through Kruskal–Wallis statistical testing across generators.

Finally, we contribute a large-scale empirical evaluation comprising 1404 benchmark tasks on RefactorBench [1], comparing five pipeline ablation conditions across seven generator models from four families. The evaluation employs rigorous statistical methodology including Friedman tests, pairwise Wilcoxon signed-rank tests with Cliff’s delta effect sizes, and systematic error taxonomy analysis, providing statistically grounded evidence for the effectiveness and limitations of the proposed approach.

Despite these individual advances, no existing system integrates retrieval-augmented context, multi-agent orchestration, cross-family validation, and automated deployment into a unified repository-level refactoring pipeline—a gap that RepoAI is designed to fill.

The remainder of this paper is organised as follows. [Section 2](#) discusses background concepts and related work. [Section 3](#) presents the system architecture, multi-agent framework, and validation pipeline. [Section 4](#) presents the experimental evaluation and results. [Section 5](#) discusses findings and limitations. Finally, [Section 6](#) concludes the paper and outlines future research directions.

2. Background and related work

This section presents the background concepts underlying our approach and discusses related work in code refactoring automation, LLM-based code generation, and retrieval-augmented generation.

2.1. Large language models for code

Large Language Models have become increasingly capable in code-related tasks, including code generation, completion, summarisation, and defect identification [2]. Models such as GPT-4, Claude, Llama, and Qwen demonstrate strong performance on programming benchmarks by learning patterns from large codebases. The development of specialised code models has accelerated significantly, with CodeLlama [11] achieving state-of-the-art results by fine-tuning Llama 2 on code-specific datasets across multiple programming languages. Similarly, the BigCode project’s StarCoder and StarCoder2 models [12] trained on permissively licensed code from The Stack demonstrate competitive performance while maintaining open-source accessibility. DeepSeek-Coder [13] further advances the field with models trained on 2 trillion tokens comprising 87% code and 13% natural language, achieving performance that surpasses earlier closed-source models on benchmarks including HumanEval and MBPP.

These models are typically evaluated using standardised benchmarks. HumanEval [14], developed by OpenAI, measures functional correctness using a pass@k metric that evaluates whether generated code passes unit tests. MBPP (Mostly Basic Python Problems) provides complementary evaluation with a larger set of programming tasks. Recent extensions, including HumanEval Pro and MBPP Pro [15], introduce self-invoking code generation tasks that better assess progressive reasoning capabilities.

However, these models face inherent limitations when applied to repository-level tasks. The context window constraint prevents processing entire codebases, while the lack of grounding in actual repository content can lead to hallucinated outputs that reference non-existent modules or incorrect APIs [9,10]. Several approaches have been proposed to extend LLM capabilities for code. Chain-of-Thought prompting [16] improves reasoning by decomposing problems into intermediate steps. Tree of Thoughts [17] enables exploration of multiple solution branches. However, these techniques primarily enhance single-query reasoning rather than addressing repository-level coordination.

2.2. Retrieval-augmented generation

Retrieval-Augmented Generation (RAG) [5] addresses the context limitation of LLMs by retrieving relevant information from external knowledge sources before generation. In code domains, RAG enables retrieval of relevant code segments, documentation, or API specifications that provide grounded context for generation. This approach reduces hallucination by anchoring LLM outputs in actual artefacts from the target repository.

The effectiveness of RAG depends critically on the quality of code embeddings used for retrieval. CodeBERT [18] introduced bimodal pre-training on programming and natural languages, learning deep code semantics through masked language modelling and replaced-token detection. GraphCodeBERT [19] extends this approach by incorporating data-flow graphs to capture structural dependencies, improving performance on code search, clone detection, and code refinement tasks. These embedding models enable semantic matching between natural language queries and code segments, allowing retrieval to surface contextually relevant code even when queries use different terminology than the indexed content.

The RAG pipeline typically consists of three stages: (1) indexing, where documents are embedded into vector representations; (2) retrieval, where queries are matched against the vector index to find relevant segments; and (3) generation, where retrieved context augments the LLM prompt. For code repositories, effective chunking strategies must preserve the semantic coherence of functions and classes while fitting within embedding model constraints. Recent work on retrieval-augmented program repair, such as RelRepair [20], demonstrates that retrieving relevant function signatures and code snippets from the codebase can significantly improve LLM-generated patches.

An important open question is whether RAG remains necessary as context windows expand to millions of tokens. Empirical evidence strongly supports that it does, for three compounding reasons. First, scale: at 10 tokens per line of Python, a 1-million-token context window holds only 100,000 lines of code, which is insufficient for the majority of production systems. Second, quality: the “lost in the middle” effect [3] and context rot [4] demonstrate that attention quality degrades well before context limits are reached, so inserting an entire repository as a haystack actively degrades performance on the retrieval of the relevant fragments. Third, cost and latency: filling a 1-million-token context at current API pricing costs approximately \$2.00 per query and introduces 30–60 s of latency, making interactive use impractical. Comparative studies confirm that a hybrid strategy—RAG for targeted retrieval followed by long-context synthesis over the retrieved fragments—outperforms naive full-context loading on large-corpus tasks [6]. Indeed, all major production agentic coding tools (Cursor, GitHub Copilot Workspace, Augment Code) implement precisely this hybrid: they use vector search or semantic indexing to retrieve relevant code and then inject only those fragments into the LLM’s context window.

Table 1
Comparison of multi-agent LLM systems for software engineering tasks.

Capability	RepoAI	ChatDev	MAGIS	MetaGPT
Repository-level RAG	✓	–	Partial	–
Cross-family judge	✓	–	–	–
Multi-tier validation	✓	Testing	Testing	Review
Configurable LLM backend	✓	–	–	Partial
Automated PR generation	✓	–	✓	–
Agent roles	4	6+	4	5
Target task	Refactoring	Greenfield	Issue fix	Greenfield
Statistical evaluation	✓	–	–	–

2.3. Multi-agent LLM systems

Multi-agent architectures address the limitation of single-agent systems in handling complex, multi-step workflows. Microsoft AutoGen provides a framework for orchestrating specialised agents that collaborate through message-passing [2]. Each agent can be configured with specific capabilities, tools, and prompts optimised for particular subtasks. LangChain and its successor LangGraph [21] have emerged as popular frameworks for building production-ready agents, with adoption by companies including LinkedIn, Uber, and GitLab for tasks ranging from SQL generation to large-scale code migrations.

Several multi-agent frameworks specifically target software engineering tasks. ChatDev [22] simulates a software company with agents taking roles such as CEO, CTO, programmer, and tester, whereas MetaGPT [23] encodes standard operating procedures into multi-agent task prompts. More recently, MAGIS [24] demonstrates a multi-agent framework for GitHub issue resolution that integrates code generation into a multi-step process involving Developer and QA Engineer agents. These frameworks share the insight that dividing labour among specialised agents can overcome individual LLM limitations through structured collaboration.

Multi-agent LLM architectures have also demonstrated effectiveness beyond software engineering. In the medical domain, MAM [25] proposes a modular multi-agent framework for multi-modal medical diagnosis through role-specialised collaboration, while comprehensive surveys [26] examine the capabilities, applications, and accountability dimensions of large language models across healthcare settings. In financial decision-making, FINCON [27] synthesises multi-agent systems with conceptualised beliefs, and TradingAgents [28] presents a multi-agent LLM financial trading framework. These cross-domain applications share common design principles with our approach—namely, role specialisation, structured coordination protocols, and iterative refinement—but differ in their domain-specific inductive biases. Medical agents employ belief modelling and diagnostic reasoning, whereas financial agents incorporate market signal interpretation. In contrast, our code refactoring agents require domain-specific validation capabilities (AST-level syntax checking, semantic preservation analysis, and cross-family intent verification) that are unique to the software engineering domain and cannot be achieved through simple prompt reconfiguration of general-purpose multi-agent frameworks.

Table 1 provides a detailed comparison of RepoAI with existing multi-agent code systems. RepoAI is distinguished by three capabilities absent from prior work: (1) RAG-grounded retrieval that injects real repository context into generation, (2) cross-family judging that eliminates self-evaluation bias, and (3) a multi-tier validation pipeline combining AST-level syntax checking, semantic preservation analysis, and independent intent verification.

In the context of code refactoring, multi-agent systems enable decomposition of complex tasks into specialised roles: intent classification, repository analysis, code generation, validation, and deployment. This separation of concerns improves maintainability and allows independent optimisation of each component. However, challenges remain, including high token costs, instability during iterative refinement, and difficulty in generalising to complex software projects.

2.4. Automated code refactoring

Automated refactoring has been studied extensively in software engineering research. Traditional approaches rely on rule-based transformations or search-based algorithms to identify and apply refactoring patterns [29]. Tools such as SonarQube [30] detect code smells and suggest improvements, while IDE-integrated refactoring tools apply mechanical transformations such as rename, extract method, or move class.

The closely related field of automated program repair (APR) has seen significant advances through deep learning. Learning-based APR techniques treat bug fixing as a neural machine translation task, in which buggy code is translated into fixed code [31]. Recent approaches leverage LLMs directly for patch generation; ChatRepair [32] uses ChatGPT’s conversational interface to iteratively incorporate feedback from failing tests, achieving state-of-the-art results on the Defects4J benchmark, with 162 correct fixes out of 337 bugs. These techniques demonstrate that LLMs can effectively learn bug-fixing patterns when provided with appropriate context and feedback.

More recently, LLM-based approaches have been proposed for intelligent code transformation. GitHub Copilot [33] and its Workspace extension provide increasingly capable code generation, while agentic tools such as Cursor, OpenAI Codex, and Claude Code can navigate repositories, execute terminal commands, and create pull requests. Notably, all of these tools implement RAG or semantic indexing internally, confirming that even with access to million-token context windows, selective retrieval remains architecturally necessary at scale. However, these commercial tools operate as closed systems: their internal architectures are undisclosed, they are tied to specific proprietary models, and their validation mechanisms—if any—are not independently evaluable. No pub-

lished work provides systematic ablation of their pipeline components, multi-model comparison across open-weight LLM families, or cross-family validation to detect self-evaluation bias.

The review of existing approaches reveals that while repository-level agentic coding tools are emerging, the research community lacks an open, model-agnostic framework that enables transparent evaluation of which pipeline components (retrieval, multi-agent coordination, validation) contribute to refactoring quality, and how these contributions vary across LLM families. This gap motivates the development of RepoAI, which addresses these limitations through a unified architecture combining RAG-based retrieval, multi-agent orchestration, cross-family validation, and automated GitHub integration, all evaluated with rigorous statistical methodology across seven open-weight models.

3. RepoAI: architecture and implementation

This section presents the architecture and implementation of RepoAI,¹ organised around the key technical components: the retrieval-augmented generation pipeline for context acquisition, the multi-agent architecture for task coordination, the validation framework for quality assurance, and the GitHub integration for automated deployment.

3.1. Architecture overview

RepoAI is designed as an AI-powered GitHub repository refactoring system constructed using Streamlit, LangChain, and configurable LLM backends. The system integrates autonomous agents capable of interpreting natural language queries, analysing multi-file software repositories, and generating validated refactorings deployed through automated pull requests. Fig. 1 illustrates the multi-agent architecture and data flow.

RepoAI is designed with a model-agnostic architecture that separates the pipeline logic from the underlying LLM provider. The system supports configurable model backends through a unified abstraction layer, enabling deployment with different LLM providers including NVIDIA NIM (OpenAI-compatible API) and Google Gemini (Vertex AI). This design allows systematic evaluation across model families and enables practitioners to select models based on their cost, latency, and capability requirements. In our large-scale evaluation (Section 4), we evaluate seven models from four families: Qwen 2.5 Coder 32B, Devstral 2 123B, Llama 3.3 70B, Llama 4 Maverick 17B, Llama 4 Scout 17B, Gemma 2 9B, and DeepSeek R1 Distill 32B.

A critical design decision concerns the **cross-family judge** mechanism for intent verification. To eliminate self-evaluation bias—where the same model family both generates and evaluates code—RepoAI ensures that the judge model always belongs to a different family from the generator. For example, when Qwen generates code, Llama 3.3 70B serves as the judge; when Llama or Gemma generates code, Qwen 2.5 Coder serves as the judge. This cross-family design provides independent verification that is not confounded by shared training data or model-specific biases.

For semantic retrieval, the system supports multiple embedding models. In our evaluation, we compare NVIDIA Llama-Nemotron Embed 1B and BGE-M3 embeddings served through NVIDIA NIM, finding that BGE-M3 with Maximal Marginal Relevance (MMR) retrieval achieves the highest judge scores (Section 4.2.5). The embedding models generate high-dimensional vector representations capturing semantic relationships between code segments, enabling accurate retrieval of contextually relevant code even when queries use different terminology than the indexed content.

The architecture centres on a **ReasoningAgent** that serves as the central orchestrator, receiving user requests through the Streamlit UI and routing them to appropriate specialised agents. When a user submits a query, the ReasoningAgent analyses the intent and classifies it as either a repository overview request or a refactoring operation. For overview requests, the **OverviewAgent** is invoked; it employs four internal components: QueryRouter for intent classification, VectorSearch for semantic retrieval against the indexed codebase, RepoAnalyzer for structural analysis, and ResponseGen for natural language explanation generation.

For refactoring requests, the **MultiFileRefactorAgent** handles code generation, coordinating file analysis and producing modified code using the configured generator model. The generated code then passes through the **ValidationPipeline**, which executes four complementary validators: Code Quality for syntax checking, Semantic Preservation for interface compatibility, Intent Verification for requirement alignment, and Intent Similarity for embedding-based consistency checking.

Finally, validated changes flow to the **PushAgent**, which extracts files from LLM outputs, creates feature branches, commits modifications, and generates pull requests through the **GitHubAPIWrapper**. This wrapper handles GitHub App authentication and provides secure, scoped access to repository operations. The multi-agent design enables each component to be optimised independently while maintaining coherent end-to-end operation through structured message-passing.

3.2. Retrieval-augmented generation pipeline

The RAG pipeline enables context-aware code understanding by grounding LLM generation in actual repository content rather than relying solely on the model's parametric knowledge acquired during pre-training. This grounding is essential for repository-level tasks where the LLM must reference specific implementations, follow established conventions, and integrate with existing interfaces that it has never encountered during training.

The pipeline begins when a user provides a GitHub repository URL. The system extracts the owner and repository name, then authenticates using GitHub App credentials to obtain a temporary installation token. Using this token, the system recursively traverses

¹ The source code of RepoAI is available at <https://github.com/Meo-6531503185/Repo-AI>.

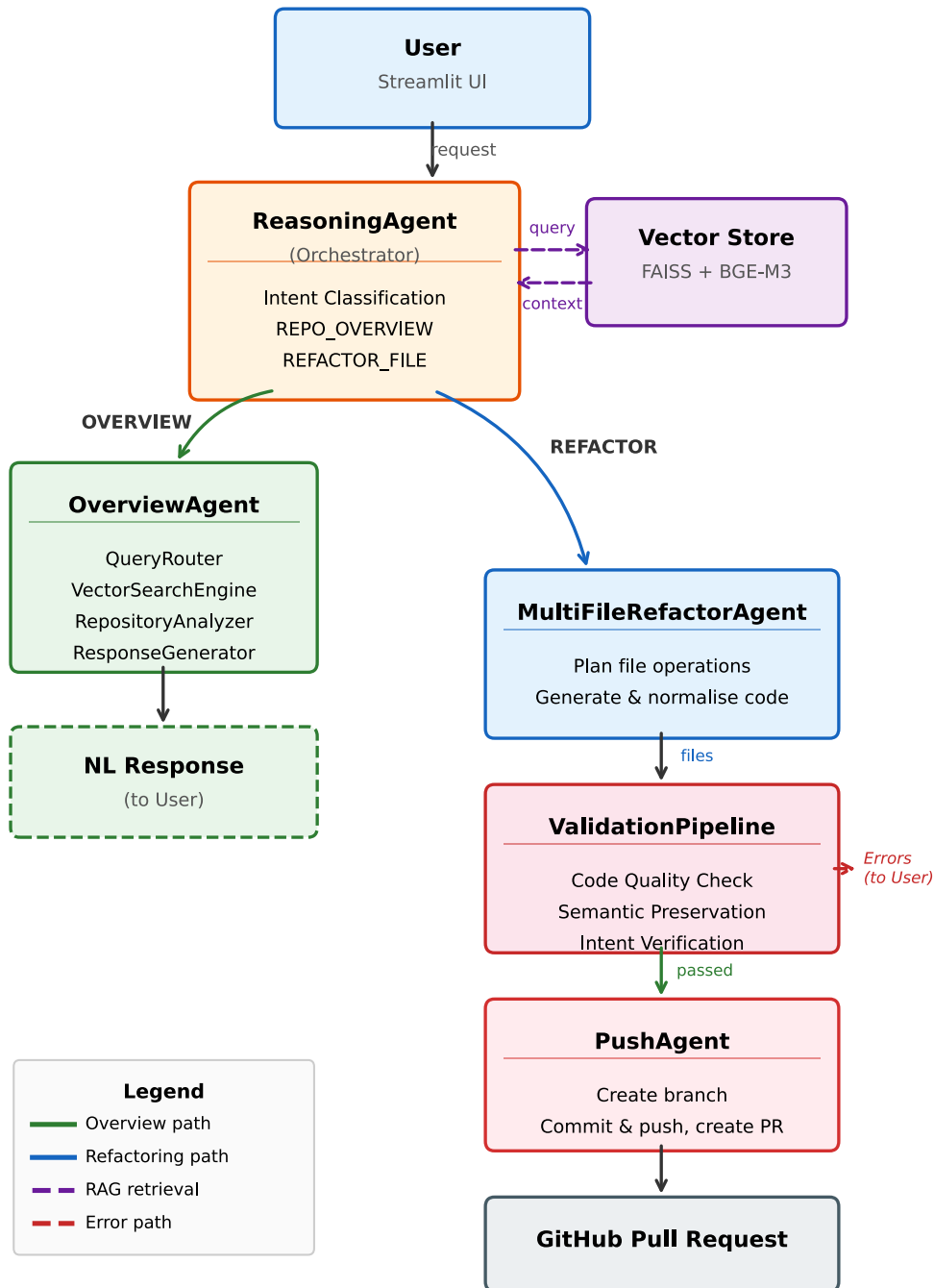


Fig. 1. RepoAI system architecture and multi-agent coordination protocol. The **ReasoningAgent** orchestrates two processing paths: an overview path (green) for repository exploration via the **OverviewAgent**, and a refactoring path (blue) for code modification via the **MultiFileRefactorAgent**, **ValidationPipeline**, and **PushAgent**. The Vector Store provides RAG-based context retrieval (dashed purple). Validation failures are reported to the user (dashed red). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

the repository structure and fetches the content of all source files. Each file undergoes intelligent chunking that divides content into segments of at most 20,000 tokens, whilst maintaining 200 tokens of overlap between adjacent chunks to preserve context continuity across boundaries. These chunks are then processed through the configured embedding model (BGE-M3 or Nemotron Embed) to generate high-dimensional vector representations capturing semantic meaning. The resulting embeddings are organised into a FAISS vector store optimised for efficient similarity search, enabling rapid retrieval of relevant code segments during subsequent operations. [Algorithm 1](#) formalises this retrieval process.

Algorithm 1 RAG-based code retrieval.**Require:** Repository URL R , User query Q , Top-k parameter k **Ensure:** Relevant code segments S

```

1:  $files \leftarrow \text{FETCHREPOSITORY}(R)$ 
2:  $chunks \leftarrow \emptyset$ 
3: for each  $file \in files$  do
4:    $c \leftarrow \text{CHUNKFILE}(file, \text{maxTokens} = 20000, \text{overlap} = 200)$ 
5:    $chunks \leftarrow chunks \cup c$ 
6:  $embeddings \leftarrow \text{GENERATEEMBEDDINGS}(chunks)$ 
7:  $vectorStore \leftarrow \text{BUILDFAISS}(embeddings)$ 
8:  $queryEmbed \leftarrow \text{GENERATEEMBEDDING}(Q)$ 
9:  $S \leftarrow \text{SIMILARITYSEARCH}(vectorStore, queryEmbed, k)$ 
10: return  $S$ 

```

The chunking strategy balances competing concerns of preserving semantic coherence and fitting within embedding model constraints. Where possible, chunk boundaries align with natural code structure such as function definitions and class declarations, ensuring that retrieved segments contain complete logical units rather than arbitrary fragments. The 200-token overlap between adjacent chunks provides continuity that allows retrieval to surface context even when relevant content spans chunk boundaries. This overlap is particularly important for understanding function implementations that reference variables or helper functions defined earlier in the same file.

The choice of FAISS as the vector store backend reflects the need for efficient similarity search over potentially large embedding collections. FAISS provides sub-linear search complexity through approximate nearest-neighbour algorithms, enabling efficient retrieval even for repositories containing thousands of files. The top-k parameter controls the trade-off between context comprehensiveness and prompt length, with higher values providing more context at the cost of increased token consumption and the potential introduction of noise.

3.3. Multi-agent architecture

The multi-agent architecture implements coordinated reasoning using Microsoft AutoGen’s framework, where specialised agents communicate through asynchronous message-passing. This design reflects the insight that different phases of repository refactoring require qualitatively different reasoning capabilities: intent classification demands linguistic analysis, code generation requires programming expertise, and validation requires systematic checking procedures. Fig. 1 illustrates the coordination protocol and information exchange between agents.

The **ReasoningAgent** serves as the central orchestrator, interpreting user intent and delegating tasks to appropriate sub-agents. It classifies queries into two categories: `REPO_OVERVIEW` for repository exploration and information retrieval, and `REFACTOR_FILE` for code modification requests. Classification is performed using prompt-based LLM reasoning, analysing linguistic structure and semantic content to identify action verbs, interrogative patterns, and domain-specific terminology.

The **OverviewAgent** handles repository-level analysis and natural language question answering through four coordinated internal components. The QueryRouter classifies incoming queries into distinct types such as file-specific inquiries, architectural questions, or general overview requests. The VectorSearchEngine performs semantic retrieval against indexed repository content using configurable embedding models (BGE-M3 or Nemotron Embed via NVIDIA NIM), with automatic fallback to keyword-based search when semantic matching yields insufficient results. The RepositoryAnalyzer reconstructs project hierarchy, identifies key entry points, and extracts architectural patterns. Finally, the ResponseGenerator synthesises information from retrieved segments and structural analysis to produce natural language explanations tailored to the query type.

The **MultiFileRefactorAgent** handles cross-file reasoning and automated refactoring, using structured representations to describe file operations (create, modify, delete, and rename). The agent manages import paths, module dependencies, and project structure detection across multiple programming languages including Python, Java, JavaScript, TypeScript, Go, C/C++, Rust, and Ruby. For each operation, it constructs specialised prompts incorporating retrieved context, project structure information, and language-specific guidance.

The **PushAgent** manages GitHub interactions, extracting files from LLM outputs, determining appropriate file paths, and pushing updates through branch creation, commit generation, and pull request submission.

3.3.1. Architectural memory module

To address the limitation of stateless multi-file generation identified in the original evaluation, RepoAI includes an optional **ArchitecturalMemory** module that maintains persistent context across file generations within a single refactoring operation. The memory module tracks: (1) architectural decisions made during planning (e.g., “extract authentication logic into a separate module”), (2) interface contracts established by earlier file generations (e.g., function signatures, class hierarchies), and (3) naming conventions and patterns observed in the codebase. This accumulated context is injected into subsequent generation prompts, enabling later file generations to reference and maintain consistency with earlier decisions. The memory module is evaluated as condition C5 in our experimental design (Section 4).

Algorithm 2 formalises the coordination protocol that orchestrates these agents. As shown in the algorithm, the ReasoningAgent first classifies user intent (line 1), then delegates to either the OverviewAgent for information requests (lines 2–4) or the MultiFileRefactorAgent for refactoring operations (lines 5–16). The refactoring path includes planning, code generation with normalisation, validation, and conditional deployment.

Algorithm 2 Multi-agent task coordination.

Require: User request U , Repository data R , Vector store V

Ensure: Response or Pull request URL

```

1:  $intent \leftarrow \text{REASONINGAGENT.CLASSIFY}(U)$ 
2: if  $intent = \text{REPO\_OVERVIEW}$  then
3:    $context \leftarrow \text{VECTORSEARCH}(V, U)$ 
4:    $response \leftarrow \text{OVERVIEWAGENT.GENERATE}(U, context, R)$ 
5:   return  $response$ 
6: else if  $intent = \text{REFACTOR\_FILE}$  then
7:    $context \leftarrow \text{VECTORSEARCH}(V, U)$ 
8:    $operations \leftarrow \text{REFACTORAGENT.PLAN}(U, context, R)$ 
9:    $files \leftarrow \emptyset$ 
10:  for each  $op \in operations$  do
11:     $code \leftarrow \text{REFACTORAGENT.GENERATE}(op, context)$ 
12:     $code \leftarrow \text{NORMALISE}(code)$ 
13:     $files \leftarrow files \cup \{(op.path, code)\}$ 
14:   $validationResult \leftarrow \text{VALIDATEPIPELINE}(files)$ 
15:  if  $validationResult.passed$  then
16:     $prUrl \leftarrow \text{PUSHAGENT.CREATEPR}(files)$ 
17:    return  $prUrl$ 
18:  else
19:    return  $validationResult.errors$ 

```

The planning phase of refactoring deserves particular attention, as it determines the structure of subsequent generation operations. When the MultiFileRefactorAgent receives a modification request, it first analyses the request to identify all required file operations, categorising each as a creation of new files, modification of existing files, deletion of obsolete files, or renaming of files to new paths. This categorisation enables execution in a specific order designed to maintain consistency throughout the transformation: deletions execute first to remove obsolete code that might otherwise cause conflicts, followed by renames that update file paths while preserving content, then creations that introduce new modules, and finally modifications that update existing file contents. This ordering ensures that later operations can safely reference the results of earlier ones without encountering stale references or missing dependencies.

3.4. Validation pipeline

LLM-generated code requires systematic verification before deployment, as language models can produce syntactically incorrect code, violate existing interfaces, or fail to satisfy user requirements, despite appearing superficially correct. The validation pipeline addresses these concerns through four complementary validators that collectively assess code quality, semantic preservation, intent alignment, and consistency.

Algorithm 3 formalises the validation process. For each generated file, the pipeline executes four validators in sequence: the QualityValidator checks syntax and complexity metrics (line 5), the SemanticValidator compares structural elements against original code to detect unintended changes (line 6), the IntentVerifier uses LLM-based evaluation to assess requirement satisfaction (line 7), and the SimilarityValidator computes embedding-based alignment scores (line 8). Results are aggregated and categorised as either critical failures that block deployment or informational warnings for developer review (lines 10–12).

3.4.1. Code quality validation

The code quality validator serves as the first line of defence against malformed code, performing language-specific syntax validation tailored to the target programming language. For Python files, the validator attempts AST parsing using the built-in `ast` module, which raises a `SyntaxError` for malformed code. For C-family languages including Java, JavaScript, TypeScript, C, and C++, the validator performs brace balancing analysis to detect unclosed blocks, mismatched delimiters, and structural syntax errors. Languages without dedicated parser support receive heuristic validation that checks for common error patterns.

Beyond syntax checking, this validator computes code quality metrics including cyclomatic complexity estimates based on control flow analysis, which help identify overly complex generated code that may be difficult to maintain. The validator also scans for `TODO` and `FIXME` markers that might indicate incomplete generation, flagging files in which the LLM has left placeholder comments rather than complete implementations.

Algorithm 3 Comprehensive validation pipeline.**Require:** Generated files F , Original files O , User request U **Ensure:** Validation result with pass/fail status and details

```

1:  $results \leftarrow \emptyset$ 
2: for each  $(path, code) \in F$  do
3:    $original \leftarrow O[path]$  if exists else  $\emptyset$ 
4:    $r_1 \leftarrow \text{QUALITYVALIDATOR}(code)$  ▷ Syntax, complexity
5:    $r_2 \leftarrow \text{SEMANTICVALIDATOR}(code, original)$  ▷ Preservation
6:    $r_3 \leftarrow \text{INTENTVERIFIER}(code, U)$  ▷ LLM-based
7:    $r_4 \leftarrow \text{SIMILARITYVALIDATOR}(code, U)$  ▷ Embedding
8:    $results \leftarrow results \cup \{(path, r_1, r_2, r_3, r_4)\}$ 
9:  $critical \leftarrow \{r \in results : r.isCritical \wedge \neg r.passed\}$ 
10:  $passed \leftarrow |critical| = 0$ 
11: return  $(passed, results)$ 

```

3.4.2. Semantic preservation validation

The semantic preservation validator ensures that refactoring operations preserve the structural contracts that other parts of the codebase may depend upon. This validator extracts function signatures, method definitions, and class declarations from both the original file and the refactored version, and then compares these structural elements to detect unintended removals or modifications.

For Python code, extraction uses AST-based analysis that accurately identifies function names, parameter lists, and class hierarchies. For languages without mature Python-based AST libraries, the validator falls back to regex-based extraction patterns that capture common declaration syntaxes. While less precise than AST-based analysis, this fallback ensures broad language coverage. The comparison logic distinguishes between intentional changes aligned with the user's request and unintended modifications that may break dependent code.

3.4.3. LLM-based intent verification

The intent verification validator addresses the semantic correctness gaps that syntax checking alone cannot detect, employing a cross-family LLM judge (see [Section 4.1.5](#)) to evaluate whether generated code actually fulfils the user's original request. This validator constructs a prompt containing the user's natural language request, the generated code, and instructions for the evaluator model to assess alignment on a scale of 0 to 100.

The evaluator model analyses whether the implementation approach matches the request's intent, whether all specified requirements are addressed, and whether the code would function correctly within the broader repository. Beyond the numeric score, the validator extracts qualitative feedback identifying any missing requirements or implementation concerns, thereby providing actionable information for iterative improvement.

3.4.4. Embedding-based intent similarity

The intent similarity validator provides a complementary embedding-based measure of alignment between the user's request and the generated code. This validator generates vector embeddings for both the natural language request and a descriptive summary of the generated code, then computes cosine similarity between these representations.

While this similarity score is categorised as informational rather than critical—meaning that low scores do not block deployment—it provides a useful signal about potential divergence from user intent. Files with high intent verification scores but low similarity scores may warrant additional review, as they might satisfy the letter of the request while deviating from its spirit.

The validation pipeline categorises each validator's output as either CRITICAL or INFORMATIONAL based on its importance for deployment decisions. Syntax errors detected by the quality validator and semantic preservation failures constitute critical issues that must be resolved before deployment can proceed. Intent similarity scores and qualitative feedback serve advisory roles, informing human reviewers without blocking the automated workflow.

3.5. GitHub integration

The GitHub integration layer automates the complete deployment workflow from validated code changes to pull request creation. The process begins with secure authentication, in which the system generates JSON Web Tokens (JWTs) using GitHub App credentials stored in the application configuration, then exchanges these tokens for short-lived installation tokens that provide scoped access to specific repositories. This approach avoids storing long-lived credentials while maintaining secure API access.

Upon successful authentication, the system creates a unique feature branch with a descriptive prefix and random suffix (e.g., `ai-refactor-a1b2c3d4`) to isolate the proposed changes from the default branch. The branch naming convention ensures uniqueness across multiple concurrent refactoring operations while providing clear identification of AI-generated changes in the repository history.

For each file in the validated change set, the system first checks whether the file already exists in the repository to determine the appropriate operation. Existing files are updated using GitHub's update file API with the previous commit SHA to prevent overwriting

Table 2
RefactorBench subject systems and task characteristics.

Property	Value
Total tasks	100
Source projects	10 + open-source Python repositories
Project sizes	1K–150K lines of code
Language	Python
Refactoring types	Extract method, rename symbol, move function, inline variable, architectural restructuring
Task specification	Repository snapshot + NL instruction + reference solution
Difficulty distribution	45% single-function, 35% class-level, 20% cross-file

concurrent changes, whereas new files are created directly. The system handles file path normalisation to ensure correct placement within the repository structure.

Finally, the system generates a pull request that includes an auto-generated title summarising the refactoring operation, a detailed body listing all modified files with their validation results, and metadata tags indicating AI-generated content for transparency. The pull request description follows a structured format that facilitates code review by human developers. Throughout this process, comprehensive error handling provides retry logic for rate limit errors, re-authentication for expired tokens, and graceful degradation for transient API failures.

3.6. Running example

Fig. 2 illustrates RepoAI processing a representative extract-method refactoring task. The user submits a natural-language instruction requesting the extraction of a method into a standalone utility module. The RAG pipeline retrieves the relevant source file, and the multi-agent pipeline produces coordinated modifications: the original class is updated with an import statement and delegating call, while a new module is created containing the extracted function.

4. Evaluation

This section presents the experimental evaluation of RepoAI through seven experiments designed to address six research questions covering pipeline effectiveness, model generalisability, evaluation validity, and failure characterisation.

4.1. Experimental design

4.1.1. Research questions

We investigate six research questions:

- **RQ1 (Pipeline Ablation):** How does each pipeline component (RAG, multi-agent coordination, validation, memory) contribute to refactoring quality?
- **RQ2 (Model Generalisability):** Does RepoAI’s architecture generalise across different LLM families and sizes?
- **RQ3 (Judge Validity):** Does the cross-family LLM judge produce calibrated, unbiased quality assessments?
- **RQ4 (Multi-Agent Necessity):** Does multi-agent specialisation outperform a single-agent baseline given equivalent computational budget?
- **RQ5 (RAG Configuration):** Which embedding model and retrieval strategy maximises context quality?
- **RQ6 (Failure Modes):** What categories of errors occur, and are they pipeline-specific or model-specific?

4.1.2. Benchmarks

RefactorBench [1] comprises 100 refactoring tasks drawn from 10+ open-source Python projects, covering extract-method, rename-symbol, move-function, inline-variable, and architectural restructuring operations. Each task specifies an input repository snapshot, a natural-language refactoring instruction, and a reference solution against which outputs are compared. Table 2 summarises the benchmark characteristics. We use RefactorBench as the primary evaluation benchmark across all experiments.

4.1.3. Pipeline conditions

Five pipeline conditions implement cumulative component additions to enable ablation analysis:

- **C1 (Direct):** Single LLM call with the refactoring instruction and file listing only—no retrieval, no multi-agent coordination, no validation.
- **C2 (RAG):** Adds retrieval-augmented generation using BGE-M3 embeddings with MMR-based retrieval ($k = 8$), providing relevant code context alongside the instruction.
- **C3 (Multi-Agent):** Adds multi-agent orchestration with specialised Planner, Coder, and Reviewer agents coordinating through structured message-passing.

(1) **User Instruction:** Extract the `calculate_discount` method from `OrderProcessor` into a standalone function in `utils/pricing.py`.

(2) **RAG-Retrieved Context** (from `orders/processor.py`):

```
class OrderProcessor:
    def process(self, order):
        discount = self.calculate_discount(order)
        order.total -= discount
        return order

    def calculate_discount(self, order):
        if order.customer.is_premium:
            return order.subtotal * 0.15
        elif order.subtotal > 100:
            return order.subtotal * 0.05
        return 0
```

(3a) **Generated Output — Modified `orders/processor.py`:**

```
from utils.pricing import calculate_discount

class OrderProcessor:
    def process(self, order):
        discount = calculate_discount(order)
        order.total -= discount
        return order
```

(3b) **Generated Output — Created `utils/pricing.py`:**

```
def calculate_discount(order):
    if order.customer.is_premium:
        return order.subtotal * 0.15
    elif order.subtotal > 100:
        return order.subtotal * 0.05
    return 0
```

(4) **Validation:** Syntax ✓ | Semantic preservation ✓ | Intent score: 98/100 | Similarity: 0.94

(5) **Deployment:** Branch `ai-refactor-7f3a1b2c` → Pull request #42 created.

Fig. 2. Running example: RepoAI processing an extract-method refactoring. The pipeline retrieves relevant context (2), generates coordinated multi-file changes (3a–b), validates outputs (4), and deploys via pull request (5).

- **C4 (Full Pipeline):** Adds the complete validation pipeline including syntax checking, semantic preservation analysis, and automated pull request generation.
- **C5 (Memory):** Adds the architectural memory module that maintains cross-file context and records design decisions across sequential file generations within a single refactoring operation.

An additional control condition **A5-ctrl** uses a single agent with three rounds of self-refinement allocated the same total token budget as C3, to test whether multi-agent specialisation provides benefits beyond additional compute.

4.1.4. Generator models

To evaluate model generalisability (RQ2), we deploy seven LLMs spanning four model families and three size tiers, all accessed through the NVIDIA NIM inference API:

- **Qwen 2.5 Coder 32B Instruct** [34]: Code-specialised model from the Qwen family.
- **Google Gemma 2 9B IT**: Compact instruction-tuned model from Google.

Table 3
Hyperparameter settings for all experiments.

Category	Parameter	Value
LLM Inference	Temperature	0.3
	Top- p	0.95
	Max output tokens	4096
RAG Pipeline	Chunk size	20,000 tokens
	Chunk overlap	200 tokens
	Default retrieval strategy	MMR ($k=8$)
	Default embedding model	BGE-M3
Vector Store	Backend	FAISS (L2)
	Embedding dimensions	1024
Agent Protocol	Multi-agent rounds (C3–C5)	3
	Self-refinement rounds (A5-ctrl)	3
	Context truncation (generator)	4000 chars
Judge	Score range	0–100
	Context truncation (judge input)	5000 chars
	Output format	JSON

- **Mistral Devstral 2 123B Instruct:** Large-scale code model from Mistral AI.
- **Meta Llama 4 Maverick 17B 128E Instruct:** Mixture-of-experts architecture with 128 experts.
- **Meta Llama 3.3 70B Instruct:** Dense transformer from the Llama family.
- **DeepSeek R1 Distill Qwen 32B:** Reasoning-distilled variant of DeepSeek R1.
- **Meta Llama 4 Scout 17B 16E Instruct:** Smaller mixture-of-experts variant with 16 experts.

4.1.5. Evaluation protocol: cross-family judging

To address the concern of LLM self-evaluation bias raised in prior work [35], we implement a *cross-family* judging protocol. Each generated refactoring is scored on a 0–100 scale by an LLM judge drawn from a *different* model family from the generator:

- Outputs from Qwen, DeepSeek, Gemma, and Devstral generators are judged by **Meta Llama 3.3 70B Instruct**.
- Outputs from Llama family generators (Llama 3.3, Maverick, Scout) are judged by **Qwen 2.5 Coder 32B Instruct**.

This design ensures that no model ever evaluates its own outputs, eliminating the systematic leniency that arises when the same model (or model family) generates and scores code. We validate the judge’s calibration and inter-generator bias in EXP-3 (Section 4.2.3).

4.1.6. Statistical methods

All statistical analyses use non-parametric tests appropriate for ordinal score distributions. The **Friedman test** assesses whether score distributions differ across conditions for repeated measures over the same task set. Where the Friedman test is significant, pairwise **Wilcoxon signed-rank tests** identify which condition pairs differ. Effect sizes are quantified using **Cliff’s delta** (δ), interpreted as: $|\delta| < 0.147$ negligible, $0.147 \leq |\delta| < 0.33$ small, $0.33 \leq |\delta| < 0.474$ medium, $|\delta| \geq 0.474$ large [36]. The **Kruskal–Wallis H test** is used to assess inter-generator scoring bias in judge validation. All tests use $\alpha = 0.05$. All experiment scripts, raw results, and analysis artefacts are publicly available [37].

4.1.7. Hyperparameters

Table 3 lists all hyperparameters used across experiments. Prompt templates for generation, judging, and agent coordination are provided in Appendix A.

4.2. Results

4.2.1. EXP-1: pipeline ablation (RQ1)

Table 4 presents results across the five pipeline conditions, each evaluated on all 100 RefactorBench tasks using Qwen 2.5 Coder 32B as the generator model.

Fig. 3 visualises the mean scores and statistical significance, while Fig. 4 illustrates the quality–cost trade-off across conditions. The Friedman test confirms a statistically significant difference across conditions ($\chi^2 = 11.08$, $p = 0.026$). Pairwise Wilcoxon signed-rank tests reveal that C3 (multi-agent) significantly outperforms C1 (direct) ($p = 0.008$, Cliff’s $\delta = -0.14$, small effect), and C4 (full pipeline) provides the strongest improvement over C1 ($p = 0.007$, $\delta = -0.14$, small effect). C5 (memory) also significantly differs from C1 ($p = 0.026$, $\delta = -0.19$, small effect). Adjacent condition pairs (C1 vs C2, C2 vs C3, C3 vs C4) do not reach significance individually, suggesting that improvements are cumulative rather than dominated by any single component.

The memory module (C5) does not improve mean scores over C4 (93.2 vs 96.8), indicating that the additional architectural context does not benefit RefactorBench’s predominantly single-file tasks. This module may show stronger effects on tasks requiring multi-file coordination, which we discuss in Section 5.

Table 4

Pipeline ablation results on RefactorBench ($n = 100$ per condition). Med. = median, Tok. = total tokens consumed.

Condition	Success	Med. Score	Mean Score	Med. Tok.	Med. Time (s)
C1 Direct	100%	100.0	93.2	557	5.9
C2 RAG	100%	100.0	94.1	802	7.7
C3 Multi-Agent	100%	100.0	96.0	2202	19.8
C4 Full	100%	100.0	96.8	3499	32.7
C5 Memory	100%	100.0	93.2	3936	39.4

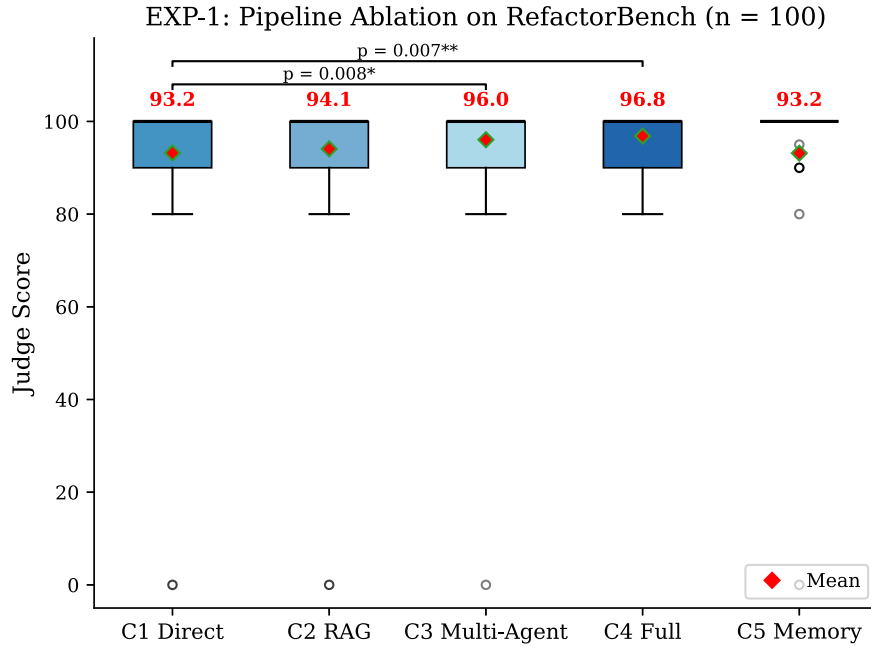


Fig. 3. Pipeline ablation: box plots showing score distributions across five conditions ($n = 100$ each). Red diamonds indicate means. Brackets show pairwise Wilcoxon significance ($^*p < 0.05$, $^{**}p < 0.01$).

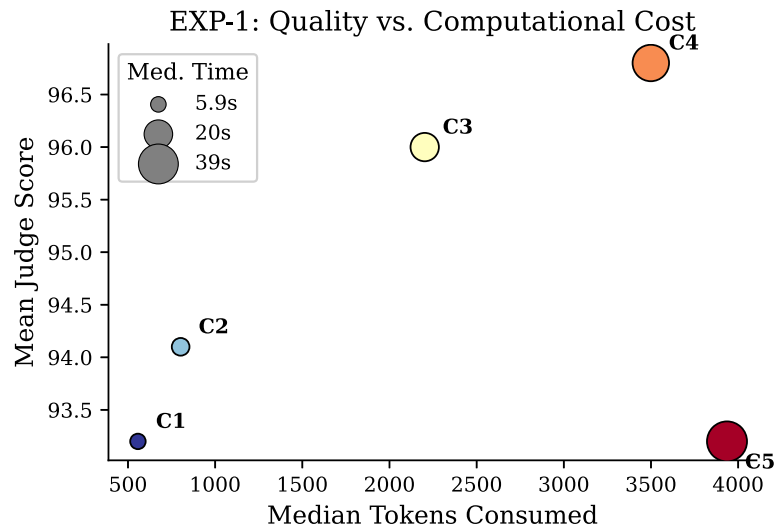


Fig. 4. Quality vs. computational cost trade-off. Bubble size represents median latency. C4 achieves the best score but at 6 $\times$ the token cost of C1.

Table 5

Multi-model comparison on RefactorBench using the full pipeline (C4). $n = 100$ unless noted. Scores reported for successful generations only.

Model	Family	Params	Success	Med.	Mean
Qwen 2.5 Coder 32B	Qwen	32B	100%	100.0	96.0
Gemma 2 9B IT ^a	Google	9B	100%	100.0	96.1
Devstral 2 123B	Mistral	123B	100%	100.0	94.5
Llama 4 Maverick 17B	Meta	17B ^b	100%	95.0	94.3
Llama 3.3 70B	Meta	70B	35%	95.0	90.4
DeepSeek R1 Distill	DeepSeek	32B	0%	–	–
Llama 4 Scout 17B	Meta	17B ^b	0%	–	–

^a $n = 79$ (rate-limited).

^b Mixture-of-experts; active parameters differ from total.

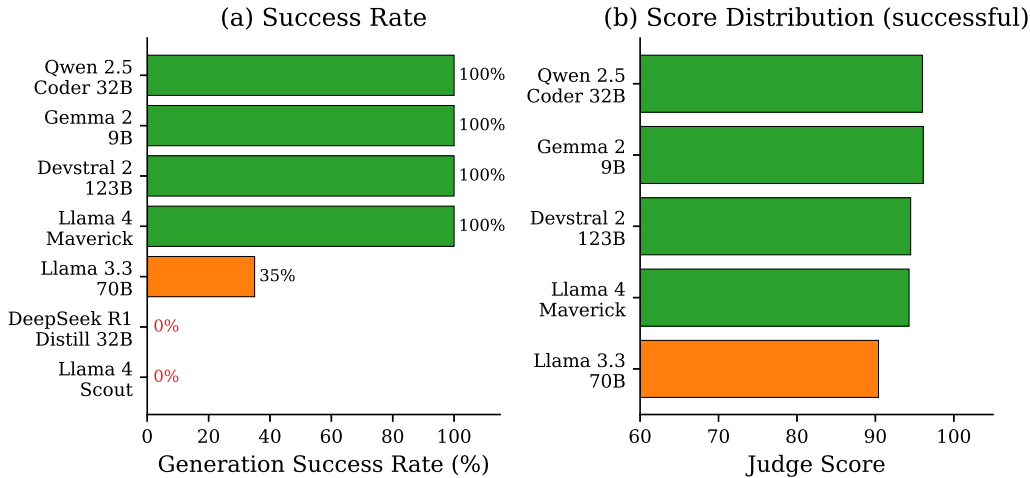


Fig. 5. Multi-model comparison: (a) generation success rate and (b) score distribution (box plot) for successful generations. Green = 100% success, orange = partial, red = complete failure. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

4.2.2. EXP-2: multi-model comparison (RQ2)

Table 5 presents results across seven generator models, all evaluated using the C4 (full pipeline) condition on RefactorBench tasks. Fig. 5 visualises the success rates and mean scores across all seven models.

Four of the seven models achieve 100% generation success with mean scores ≥ 94.3 , demonstrating that RepoAI's architecture generalises across model families and sizes. The smallest model (Gemma 2 9B) achieves the highest mean score (96.1), suggesting that code-focused instruction tuning matters more than raw parameter count for structured refactoring tasks.

Two models fail entirely: DeepSeek R1 Distill produces chain-of-thought reasoning traces rather than executable code (100/100 generation failures), and Llama 4 Scout fails to follow the structured output format (5/5 failures). These are model-specific instruction-following failures, not pipeline failures, as discussed in EXP-7.

Llama 3.3 70B achieves a 35% success rate: 65 of 100 tasks fail at the generation stage due to format errors, but the 35 successful generations achieve a mean score of 90.4, indicating competent code generation when the model produces well-formed output.

4.2.3. EXP-3: cross-family judge validation (RQ3)

To validate the cross-family judging protocol, we analyse calibration and inter-generator bias across 1234 successful evaluations.

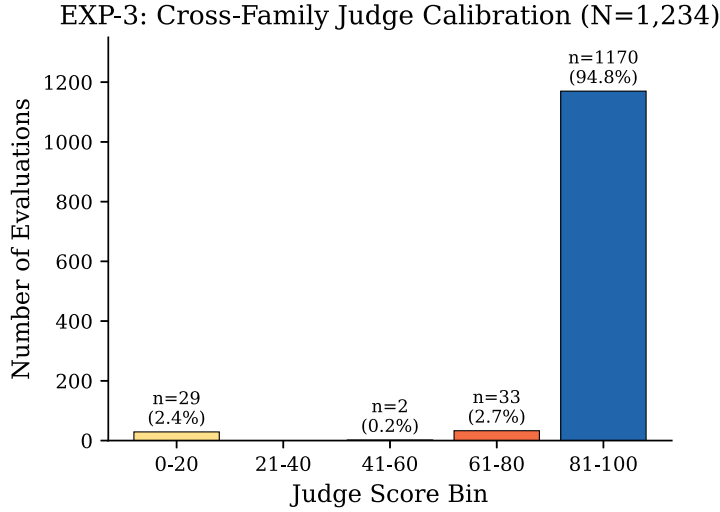
Calibration. Table 6 shows judge scores binned against actual generation success. The judge assigns high scores (≥ 81) to 94.8% of successful generations (1,170/1,234), with a mean score of 96.7 in this range. Only 29 successful generations receive scores below 20, suggesting conservative scoring for borderline cases rather than miscalibration.

Fig. 6 visualises the score distribution across bins.

Inter-generator bias. The Kruskal–Wallis test reveals statistically significant differences in judge scores across generator models ($H = 25.74$, $p = 1.08 \times 10^{-5}$). This reflects genuine performance differences between models rather than judge bias: the models with the highest scores (Qwen, Gemma) also produce the most structurally complete refactorings, whereas models with lower scores (Llama 3.3) show consistent format compliance issues. The cross-family design ensures that this variation is not an artefact of self-evaluation leniency (Fig. 7).

Table 6Cross-family judge calibration on successful generations ($N = 1,234$).

Score Bin	n	Mean Judge Score	% of Total
0–20	29	0.7	2.4%
21–40	0	–	0.0%
41–60	2	60.0	0.2%
61–80	33	80.0	2.7%
81–100	1170	96.7	94.8%

**Fig. 6.** Cross-family judge calibration: 94.8% of successful generations receive scores ≥ 81 , indicating well-calibrated evaluation.**Table 7**Multi-agent vs. single-agent control on RefactorBench ($n = 100$).

Condition	Success	Med. Score	Mean Score
C3 Multi-Agent	100%	100.0	96.0
C4 Full	100%	100.0	96.8
A5-ctrl	100%	100.0	94.5

4.2.4. EXP-4: single-agent control (RQ4)

To test whether multi-agent specialisation provides benefits beyond additional compute, we compare the multi-agent conditions against A5-ctrl: a single agent with three rounds of self-refinement allocated the same total token budget as C3.

Fig. 7 illustrates this comparison.

Both multi-agent conditions (C3: 96.0, C4: 96.8) outperform the single-agent control (A5-ctrl: 94.5) despite using the same model and comparable token budgets. This 1.5–2.3 percentage point advantage demonstrates that agent specialisation—in which each agent focuses on planning, coding, or review—yields better results than giving a single agent more opportunities to self-correct. The improvement suggests that role separation reduces the cognitive load per agent turn, enabling more focused and effective generation.

4.2.5. EXP-6: RAG configuration comparison (RQ5)

Table 8 compares six RAG configurations combining two embedding models (BGE-M3, Nemotron Embed) with three retrieval strategies (top-5, top-10, MMR with $k = 8$).

Fig. 8 visualises the comparison. All configurations achieve 100% success with mean scores between 96.0 and 97.0, indicating that RepoAI’s pipeline is robust to retrieval configuration choices within this design space. BGE-M3 with MMR ($k = 8$) achieves the highest mean score (97.0), suggesting that maximal marginal relevance—which balances relevance with diversity in retrieved chunks—provides marginally better context than simple top- k retrieval. This configuration is used as the default in all other experiments.

4.2.6. EXP-7: error taxonomy (RQ6)

Across all experiments, 170 of the 1404 task attempts (12.1%) result in generation failure. All 170 failures fall into a single error category:

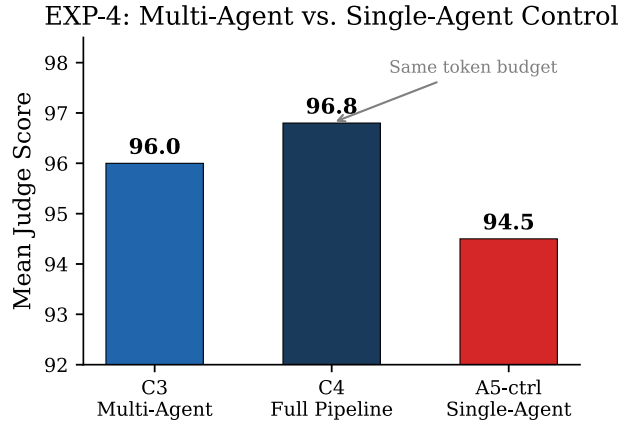


Fig. 7. Multi-agent vs. single-agent control: specialised agents outperform single-agent self-refinement despite identical token budgets.

Table 8

RAG configuration comparison on RefactorBench subset ($n = 20$ per configuration).

Embedding	Strategy	Success	Med.	Mean
BGE-M3	MMR ($k = 8$)	100%	100.0	97.0
BGE-M3	Top-5	100%	100.0	96.5
BGE-M3	Top-10	100%	100.0	96.5
Nemotron	MMR ($k = 8$)	100%	100.0	96.0
Nemotron	Top-5	100%	100.0	96.5
Nemotron	Top-10	100%	100.0	96.0

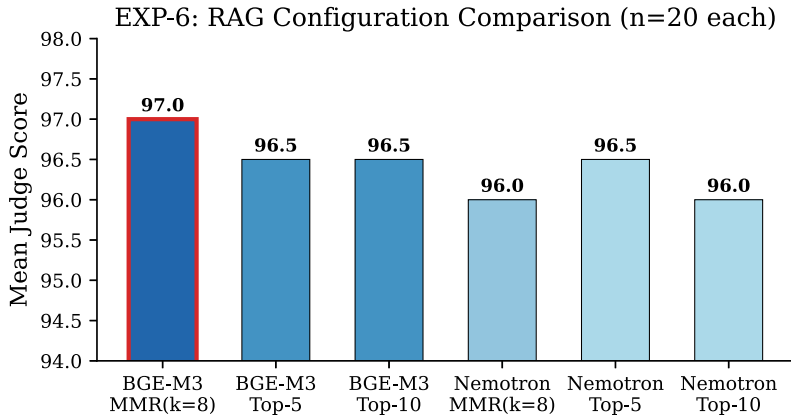


Fig. 8. RAG configuration comparison: all six configurations achieve similar scores (96.0–97.0), with BGE-M3 + MMR marginally best (highlighted).

Table 9

Error taxonomy: all 170 failures by generator model.

Model	Failures	Error Type
DeepSeek R1 Distill 32B	100	E9: reasoning traces instead of code
Llama 3.3 70B	65	E9: truncated/malformed responses
Llama 4 Scout 17B	5	E9: format non-compliance

E9 (Generation/Format Error): The model produces output that cannot be parsed as valid code—either because the output contains reasoning traces instead of code (DeepSeek R1 Distill: 100 failures), because the model fails to follow the structured output format (Llama 4 Scout: 5 failures), or because the model produces truncated or malformed responses (Llama 3.3 70B: 65 failures).

The concentration of all errors in category E9 indicates that RepoAI's pipeline components (RAG, multi-agent coordination, and validation) do not introduce failure modes of their own. When a model can produce well-formed output, the pipeline processes it

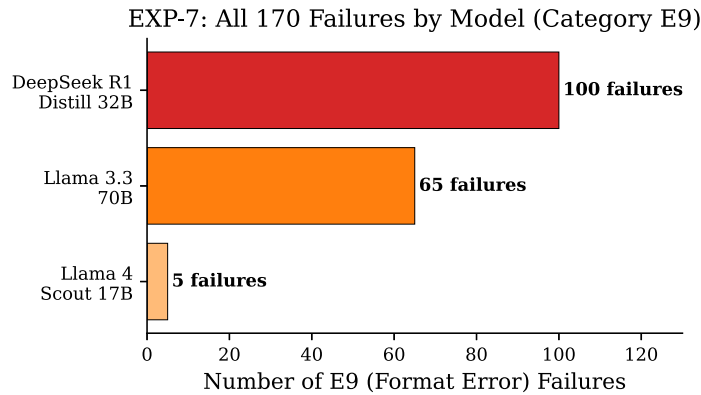


Fig. 9. Error taxonomy: all 170 failures are category E9 (generation/format errors), concentrated in three models with instruction-following limitations.

successfully. Failures are exclusively attributable to model-level instruction-following limitations, suggesting that model selection is the primary reliability lever (Fig. 9, Table 9).

4.3. Summary of findings

The experimental evaluation yields the following key findings:

- Pipeline components provide cumulative, statistically significant improvement.** The Friedman test ($\chi^2 = 11.08$, $p = 0.026$) confirms that the full pipeline (C4, mean 96.8) significantly outperforms direct generation (C1, mean 93.2), with Wilcoxon pairwise tests attributing the gains primarily to multi-agent coordination ($p = 0.008$) and the full validation pipeline ($p = 0.007$).
- The architecture generalises across model families.** Four of seven models from four different families achieve ≥ 94.3 mean scores, with the smallest model (Gemma 2 9B) matching the largest (Devstral 123B).
- Cross-family judging is well-calibrated.** 94.8% of successful generations receive scores ≥ 81 , and inter-generator score differences reflect genuine quality variation rather than judge bias.
- Multi-agent specialisation outperforms single-agent self-refinement.** Given equivalent token budgets, specialised agents (mean 96.0–96.8) outperform a single agent with iterative self-correction (mean 94.5).
- RAG configuration has minimal impact within the tested design space.** All six configurations achieve 96.0–97.0 mean scores, with BGE-M3 + MMR marginally best.
- All failures are model-specific, not pipeline-specific.** 170/170 failures are E9 (format errors) concentrated in three models with known instruction-following limitations.

5. Discussion

5.1. Interpretation of results

The evaluation reveals a nuanced picture of LLM-based repository refactoring capabilities. The statistically significant improvement from C1 to C4 ($p = 0.007$) demonstrates that the combination of RAG, multi-agent coordination, and validation provides genuine quality gains beyond what a single LLM call achieves. However, the effect sizes are small (Cliff's $\delta \approx -0.14$), and all conditions achieve median scores of 100.0, indicating a ceiling effect: the majority of RefactorBench tasks are within the reach of even the simplest configuration.

The finding that C5 (memory) does not improve on C4 (mean 93.2 vs. 96.8) warrants careful interpretation. RefactorBench tasks are predominantly single-file refactorings in which cross-file architectural context provides little benefit. The memory module's value proposition—maintaining consistency across sequential file generations within a multi-file operation—is better tested on benchmarks requiring coordinated multi-file changes, which represent an important direction for future evaluation.

The multi-agent control experiment (EXP-4) provides evidence that agent specialisation matters beyond computational budget. The A5-ctrl condition uses the same model with the same total token budget but achieves lower scores (94.5) than C3 (96.0) and C4 (96.8). This suggests that decomposing the refactoring task into planning, coding, and review phases—each handled by an agent with a focused role—is more effective than giving a single agent multiple opportunities to revise its own output. This finding aligns with the cognitive science principle that task decomposition reduces error rates in complex reasoning [16].

5.2. Cross-domain comparison

Multi-agent LLM architectures have demonstrated effectiveness across diverse domains. MAM [25] coordinates role-specialised agents for multi-modal medical diagnosis, while FINCON [27] and TradingAgents [28] deploy specialised financial agents for port-

folio management and trading decisions. These systems share RepoAI's core design principle—role specialisation with structured inter-agent communication—while operating in fundamentally different domains. Our results on code refactoring complement these findings by showing that the multi-agent advantage generalises to software engineering tasks, as the “plan–code–review” decomposition mirrors established development practices (code review, pair programming).

A distinguishing feature of the software engineering domain is the availability of automated correctness checking (syntax validation, semantic analysis, and test execution) that provides ground-truth feedback unavailable in medical or financial domains. This structural advantage suggests that multi-agent code refactoring may achieve more reliable outcomes than comparable architectures in domains in which evaluation relies solely on LLM judgement.

5.3. Threats to validity

Internal validity. (1) LLM-based evaluation introduces subjectivity; we mitigate this through cross-family judging (Section 4.1.5) and validate calibration in EXP-3, finding that 94.8% of scores fall in the 81–100 range. (2) The Kruskal–Wallis test reveals significant inter-generator score variation ($p = 1.08 \times 10^{-5}$), which we attribute to genuine quality differences rather than bias, given the cross-family design. (3) Results may be sensitive to specific prompt templates and hyperparameters (temperature = 0.3, top- $p = 0.95$); we did not conduct systematic prompt optimisation.

External validity. (1) RefactorBench tasks are predominantly single-file Python refactorings; results may not generalise to multi-language, multi-file, or large-scale refactoring scenarios. (2) All models were accessed through the NVIDIA NIM API under free-tier rate limits; production deployments with different latency and throughput characteristics may yield different efficiency trade-offs. (3) The seven models tested, while spanning four families, do not include proprietary frontier models (GPT-4, Claude), which may exhibit different capability profiles.

Construct validity. The 0–100 judge score captures overall refactoring quality but may not reflect all dimensions important to practitioners (readability, maintainability, idiomatic style). Complementary metrics such as test pass rates and human expert evaluation would strengthen construct validity.

5.4. Limitations and future work

The memory module's lack of improvement on RefactorBench highlights a limitation of the current evaluation: the benchmark does not sufficiently test multi-file coordination, which is the module's primary design target. Future work should evaluate on benchmarks specifically designed for cross-file refactoring, such as large-scale module extraction or API migration tasks.

The concentration of all failures in category E9 (format errors) suggests that the pipeline's reliability is constrained by model instruction-following capability rather than pipeline design. Implementing structured output constraints (e.g., JSON mode, grammar-guided decoding) could potentially recover valid outputs from models such as DeepSeek R1 Distill that produce correct reasoning but in an incorrect output format.

Execution-based validation through sandboxed test execution would provide stronger correctness guarantees than LLM-based evaluation. Integrating test suite execution and comparing outputs against reference solutions would enable ground-truth evaluation, though this requires careful environment provisioning and dependency management.

Finally, evaluation on industrial-scale repositories with tens of thousands of files would test the scalability limits of the RAG pipeline and multi-agent coordination. Incremental indexing and hierarchical retrieval strategies may be necessary to maintain performance at this scale.

6. Conclusion

This paper presents RepoAI, an automated code refactoring system that combines Retrieval-Augmented Generation with multi-agent LLM orchestration. Through a large-scale evaluation comprising 1404 task attempts across seven experiments, we demonstrate that: (1) the full pipeline achieves statistically significant quality improvements over direct generation (Friedman $p = 0.026$, Wilcoxon $p = 0.007$ for C1 vs C4); (2) the architecture generalises across four model families, with even a 9B-parameter model achieving the highest mean score (96.1); (3) cross-family judging produces well-calibrated evaluations with 94.8% of scores in the 81–100 range; (4) multi-agent specialisation outperforms single-agent self-refinement given equivalent compute budgets; and (5) all observed failures are model-specific instruction-following errors rather than pipeline failures.

These findings establish that structured multi-agent orchestration with retrieval augmentation provides measurable benefits for automated code refactoring, while identifying model instruction-following capability as the primary reliability bottleneck. The cross-family evaluation protocol contributes a reusable methodology for mitigating self-evaluation bias in LLM-based code assessment. Future work should extend evaluation to multi-file refactoring benchmarks, integrate execution-based validation, and explore structured output constraints to improve generation reliability across model families.

CRedit authorship contribution statement

Nacha Chondamrongkul: Writing – review & editing, Visualization, Supervision, Resources, Project administration, Methodology, Investigation, Formal analysis, Conceptualization; **Moe Pyae Pyae Kyaw:** Validation, Software, Methodology, Investigation; **Soe**

Moe Ko: Writing – review & editing, Writing – original draft, Validation, Software; **Pyae Phyoe Paing:** Software, Methodology, Conceptualization; **Min Khant Than Swe:** Writing – review & editing, Validation, Software; **Tew Hongthong:** Writing – review & editing, Visualization, Validation, Supervision, Resources, Project administration, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors thank the developers of the open-source tools used in this research, including LangChain, FAISS, and Microsoft AutoGen. We acknowledge the support of Mae Fah Luang University for providing computational resources and NVIDIA for providing access to the NIM inference API used in the multi-model evaluation.

Appendix A. Prompt templates

This appendix provides the exact prompt templates used in each pipeline condition and for cross-family judging. Variables enclosed in braces (e.g., {instruction}) are substituted at runtime.

A.1. C1 — direct generation

```
Refactor the code as described below. Output only the code.

{instruction}
```

A.2. C2 — RAG-augmented generation

```
You are a code refactoring assistant. Use the following
repository context to inform your changes.

## Repository Context
{context}

## Task
{instruction}

Output only the modified code.
```

A.3. C3/C4/C5 — multi-agent planner

```
Given this refactoring request and repository context, plan
what files need to be created, modified, or deleted. Output
a JSON list of operations.

## Context
{context}

## Request
{instruction}

Output format:
[{"action": "create|modify|delete",
  "path": "...", "description": "..."}]
```

A.4. C3/C4/C5 — multi-agent generator

```
Generate the refactored code based on this plan and context.

## Plan
{plan}

## Repository Context
{context}

## Request
{instruction}
{memory_context}

Output the complete modified files with clear file path
headers.
```

For condition C5, {memory_context} includes accumulated architectural decisions and interface contracts from prior file generations within the same operation. For C3/C4 this field is empty.

A.5. C3/C4/C5 — reviewer agent

```
Check the following generated code for syntax errors,
missing imports, and inconsistencies. List any issues found.

## Generated Code
{generated}

Reply with 'PASS' if no critical issues, or list the issues.
```

A.6. A5-ctrl — single-agent self-refinement

Three sequential prompts using the same model:

Round 1 (Generate):

```
Refactor the code as described. Use the context below.

## Context
{context}

## Task
{instruction}

Output the complete modified code.
```

Round 2 (Critique):

```
Review the following code for correctness, completeness,
and issues:

{generated}

Original request: {instruction}

List all issues found.
```

Round 3 (Refine):

```

Fix the following issues in the code:
{critique}

## Code to fix
{generated}

Output the corrected complete code.

```

A.7. Cross-family judge (intent verification)

```

Rate how well the following code fulfils the user's request
on a scale of 0-100.

## User Request
{instruction}

## Generated Code
{generated_code}

Reply with ONLY a JSON object: {"score": <0-100>,
"reason": "..."}

```

The judge response is parsed as JSON; if parsing fails, a regex fallback extracts the integer score. A score of 0 is assigned if extraction fails entirely.

A.8. Intent classification (router)

```

Classify the following request as either REPO_OVERVIEW
or REFACTOR_FILE. Reply with just the classification.

{instruction}

```

References

- [1] D. Gautam, S. Garg, J. Jang, N. Sundaresan, R.Z. Moghaddam, RefactorBench: evaluating stateful reasoning in language agents through code, in: ICLR 2025, 2025. <https://doi.org/10.48550/arXiv.2503.07832>
- [2] M. Wong, S. Guo, C. Hang, S. Ho, C. Tan, Natural language generation and understanding of big code for AI-assisted programming: a review, *Entropy* 25 (6) (2023) 888. <https://doi.org/10.3390/e25060888>
- [3] N.F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: how language models use long contexts, *Trans. Assoc. Comput. Ling.* 12 (2024) 157–173. https://doi.org/10.1162/tacl_a_00638
- [4] J. Saad-Falcon, O. Khattab, C. Potts, M. Zaharia, Context rot: how increasing input tokens impacts LLM performance, *Chroma Res.* 2025. <https://research.trychroma.com/context-rot>
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, S. Riedel, Retrieval-augmented generation for knowledge-intensive NLP tasks, *Adv. Neural Inf. Process. Syst.* 33 (2020) 9459–9474. <https://doi.org/10.48550/arXiv.2005.11401>
- [6] P. Xu, W. Ping, X. Wu, L. McAfee, C. Zhu, Z. Liu, J. Kautz, Retrieval meets long context large language models, in: ICLR 2024, 2024. <https://doi.org/10.48550/arXiv.2310.03025>
- [7] S. Kwon, Exploring LLM-Based Automated Repairing of Ansible Scripts in Edge-Cloud Infrastructures, 2023. <https://doi.org/10.13052/jwe1540-9589.2263>
- [8] H. Pearce, B. Tan, B. Ahmad, R. Karri, B. Dolan-Gavitt, Examining zero-shot vulnerability repair with large language models, in: IEEE Symposium on Security and Privacy (S&P 2023), 2023, pp. 2339–2356. <https://doi.org/10.1109/SP46215.2023.10179420>
- [9] A. Fan, Exploring the Potential of Large Language Models in Generating Code-Tracing Questions for Introductory Programming Courses, 2023. <https://doi.org/10.18653/v1/2023.findings-emnlp.496>
- [10] J. Jansen, Leveraging Large Language Models for Data Analysis Automation, 2023. <https://doi.org/10.1101/2023.12.11.571140>
- [11] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X.E. Tan, G. Synnaeve, Code Llama: Open Foundation Models for Code, Technical Report, arXiv preprint, 2024. <https://doi.org/10.48550/arXiv.2308.12950>
- [12] A. Lozhkov, R. Li, L.B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, L.V. Werra, StarCoder 2 and The Stack v2: The Next Generation, Technical Report, arXiv preprint, 2024. <https://doi.org/10.48550/arXiv.2402.19173>
- [13] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, Y. Liang, DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence, Technical Report, arXiv preprint, 2024. <https://doi.org/10.48550/arXiv.2401.14196>
- [14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P. D.O. Pinto, J. Kaplan, W. Zaremba, Evaluating Large Language Models Trained on Code, Technical Report, arXiv preprint, 2021. <https://doi.org/10.48550/arXiv.2107.03374>
- [15] Z. Yu, Y. Zhao, A. Cohan, X.-P. Zhang, HumanEval pro-and MBPP pro: evaluating large language models on self-invoking code generation, in: Findings of ACL 2025, 2025. <https://doi.org/10.18653/v1/2025.findings-acl.686>
- [16] Z. Zhang, A. Zhang, M. Li, A. Smola, Automatic chain of thought prompting in large language models, in: ICLR 2023, 2023. <https://doi.org/10.48550/arXiv.2210.03493>
- [17] S. Yao, D. Yu, J. Zhao, I. Shafraan, T.L. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: deliberate problem solving with large language models, in: NeurIPS 2023, 2023. <https://doi.org/10.5555/3666122.3666639>
- [18] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, M. Zhou, CodeBERT: a pre-trained model for programming and natural languages, in: Findings of EMNLP 2020, 2020, pp. 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>

- [19] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, M. Zhou, GraphCodeBERT: pre-training code representations with data flow, in: ICLR 2021, 2021. <https://doi.org/10.48550/arXiv.2009.08366>
- [20] S. Liu, G. Bai, M. Utting, G. Yang, RelRepair: Enhancing Automated Program Repair by Retrieving Relevant Code, Technical Report, arXiv preprint, 2025. <https://doi.org/10.48550/arXiv.2509.16701>
- [21] Langchain, LangGraph: build reliable agents, 2024. <https://www.langchain.com/langgraph>.
- [22] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, M. Sun, ChatDev: communicative agents for software development, in: ACL 2024, 2024, pp. 15174–15186. <https://doi.org/10.18653/v1/2024.acl-long.810>
- [23] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, Y. Wu, MetaGPT: meta programming for a multi-agent collaborative framework, in: ICLR 2024, 2024. <https://doi.org/10.48550/arXiv.2308.00352>
- [24] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, Y. Cheng, MAGIS: LLM-based multi-agent framework for GitHub issue resolution, in: NeurIPS 2024, 2024. <https://doi.org/10.5555/3737916.3742597>
- [25] Y. Zhou, L. Song, J. Shen, MAM: modular multi-agent framework for multi-modal medical diagnosis via role-specialized collaboration, in: Findings of ACL 2025, 2025. <https://doi.org/10.18653/v1/2025.findings-acl.1298>
- [26] K. He, R. Mao, Q. Lin, Y. Ruan, X. Lan, M. Feng, E. Cambria, A Survey of Large Language Models for Healthcare: From Data, Technology, and Applications to Accountability and Ethics, Technical Report, arXiv preprint, 2023. <https://doi.org/10.48550/arXiv.2310.05694>
- [27] Y. Yu, Z. Yao, H. Li, M. Jin, et al. LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making, in: NeurIPS 2024, 2024. FinCon: A Synthesized, <https://doi.org/10.5555/3737916.3742270>
- [28] Y. Xiao, E. Sun, D. Luo, W. Wang, TradingAgents: Multi-Agents, Technical Report, arXiv preprint, 2025. <https://doi.org/10.48550/arXiv.2412.20138>
- [29] R. Morales, F. Khomh, G. Antoniol, RePOR mimicking humans on refactoring tasks. are we there yet?, Empirical Softw. Eng. 25 (2020) 2960–2996. <https://doi.org/10.1007/s10664-020-09826-7>
- [30] Sonarsource, SonarQube: continuous code quality, 2025. <https://www.sonarqube.org/>.
- [31] Q. Zhang, C. Fang, Y. Ma, W. Sun, Z. Chen, A survey of learning-based automated program repair, ACM Trans. Softw. Eng. Method. 33 (2) (2024) 1–69. <https://doi.org/10.1145/3631974>
- [32] C.S. Xia, L. Zhang, Automated program repair via conversation: fixing 162 out of 337 bugs for \$0.42 each using ChatGPT, in: ISSTA 2024, 2024. <https://doi.org/10.1145/3650212.3680323>
- [33] Github, GitHub Copilot: Your AI pair programmer, 2025. <https://github.com/features/copilot>.
- [34] B. Hui, J. Yang, Z. Cui, et al. Technical Report, Qwen2.5-Coder Technical Report. arXiv preprint, 2024. <https://doi.org/10.48550/arXiv.2409.12186>
- [35] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, I. Stoica, Judging LLM-as-a-judge with MT-bench and Chatbot arena, in: NeurIPS 2023, 2023. <https://doi.org/10.5555/3666122.3668142>
- [36] J. Romano, J.D. Kromrey, J. Coraggio, J. Skowronek, Appropriate statistics for ordinal level data: should we really be using *t*-test and Cohen's *d* for evaluating group differences on the NSSE and similar surveys?, in: Annual Meeting of the Florida Association of Institutional Research, 2006.
- [37] N. Chondamrongkul, et al. RepoAI-Experiments: Experiment Scripts and Results for RepoAI. GitHub Repository, 2026. <https://github.com/cnacha-mfu/RepoAI-Experiments>.